

DSA COACH

[HackerRank — Simple, Medium & Hard Problem Coaching System]

Project Name:	DSA-Coach-HackerRank (Simple + Medium + Hard Problems)
Primary Objective:	To build an AI-powered DSA coaching system that helps learners practice and understand HackerRank-style problems across Simple, Medium, and Hard difficulty levels.
Domain:	Artificial Intelligence & Machine Learning / Generative AI / Agentic AI / Education Technology / Coding Practice
Core Focus:	DSA problem solving, personalized guidance, hints, explanations, complexity analysis, and progressive difficulty.

Project concept: A virtual DSA coach that organizes coding practice by difficulty and uses AI to guide the learner from problem understanding to solution strategy.

Technical Project Documentation

1. Name

Project Name: DSA-Coach-HackerRank (Simple + Medium + Hard Problems)

Primary Objective: To develop an intelligent DSA learning and coding-practice assistant that helps users solve HackerRank-style problems at Simple, Medium, and Hard levels through explanations, hints, approach suggestions, and feedback.

2. Problem Statement

Students preparing for coding assessments often solve large numbers of DSA problems without a structured learning path. They may struggle to identify the correct data structure, algorithm, complexity, or next step when they are stuck. The DSA Coach project provides a guided practice environment where problems can be organized by difficulty and the learner can receive AI-assisted coaching instead of only seeing a final answer.

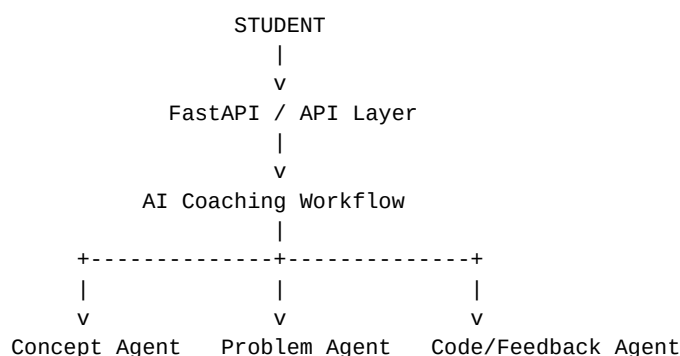
3. Project Objectives

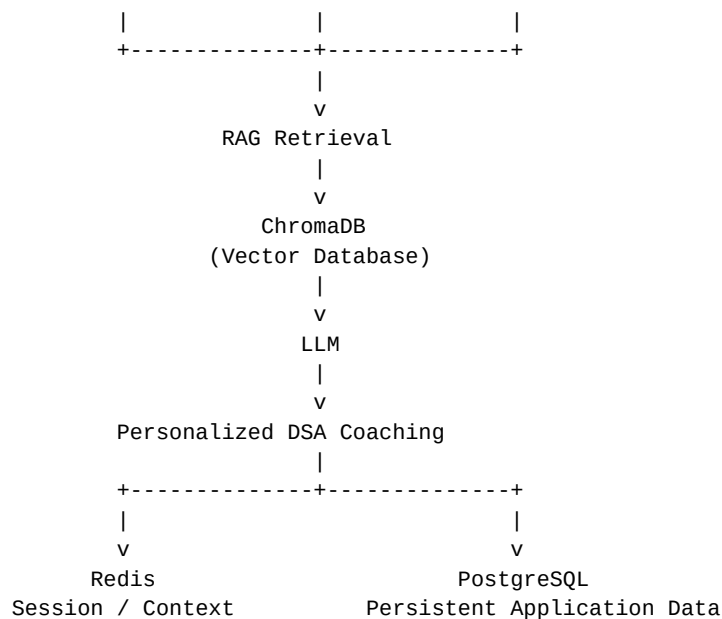
- Provide DSA problems in three progressive levels: Simple, Medium, and Hard.
- Help learners understand problem statements before attempting implementation.
- Provide hints progressively so the learner can continue solving independently.
- Explain suitable algorithms and data structures for a problem.
- Discuss time and space complexity.
- Support personalized AI-based coaching and feedback.
- Use Agentic AI to coordinate specialized coaching tasks.
- Use RAG and ChromaDB to retrieve relevant DSA knowledge when configured.
- Provide a scalable backend for the coaching application.

4. Difficulty Model

Level	Purpose	Typical Coaching
Simple	Build fundamentals and confidence	Concept explanation, basic hint, straightforward approach
Medium	Develop algorithmic reasoning	Approach comparison, complexity analysis, guided debugging
Hard	Develop advanced problem-solving abilities	Deeper reasoning, optimization hints, advanced patterns

5. System Architecture





6. Agentic AI in the Project

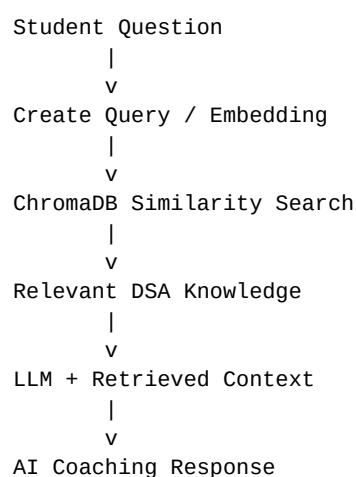
Agentic AI is used to make the DSA coach more than a simple chatbot. Specialized agents can perform different parts of the coaching process and the workflow can coordinate them according to the learner's request.

- **Problem Understanding Agent:** Helps interpret the problem and identify the required task.
- **DSA Concept Agent:** Identifies the relevant data structure, algorithm, or problem-solving pattern.
- **Hint/Coaching Agent:** Gives progressive hints without unnecessarily revealing the complete solution.
- **Code/Feedback Agent:** Reviews a learner's approach or code when that functionality is implemented.
- **Complexity Guidance:** Explains expected time and space complexity and possible optimization.
- **Coordinator:** Routes the request through the appropriate agent(s) and combines the resulting context.

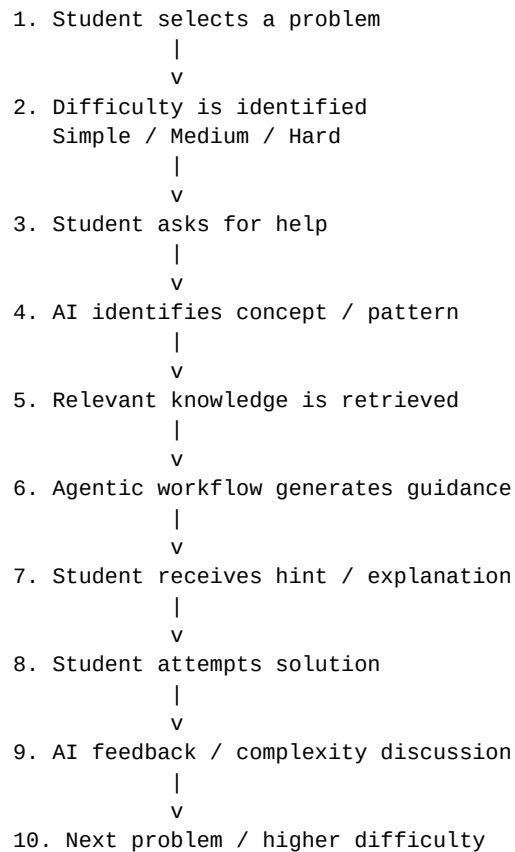
Important: The exact agent names and workflow should match the agents implemented in the repository.

7. RAG and ChromaDB

Retrieval-Augmented Generation (RAG) can be used to ground the AI coach with the project's DSA knowledge base. Relevant material is retrieved before the LLM generates the coaching response.



8. Problem Coaching Workflow



9. Example User Interaction

Student: "I am stuck on this Medium array problem. I know the brute-force method, but it is too slow."

Coach: The system can identify the array/sliding-window or prefix-sum pattern when supported by the knowledge base, retrieve relevant guidance, and provide a progressive hint such as what information should be maintained while iterating. It can then discuss why the optimized approach improves the complexity.

10. DSA Topics Covered

- Arrays and Strings
- Searching and Sorting
- Two Pointers and Sliding Window
- Hashing / Hash Maps
- Stacks and Queues
- Linked Lists
- Recursion and Backtracking
- Trees and Binary Search Trees
- Heaps / Priority Queues
- Graphs and Graph Traversal
- Greedy Algorithms
- Dynamic Programming

- Bit Manipulation
- Time and Space Complexity

11. Technology Stack

Component	Technology / Concept
Domain	AI/ML + Generative AI + Agentic AI + EdTech
Backend	FastAPI
Agent Framework	CrewAI
Retrieval	RAG
Vector Database	ChromaDB
Session Management	Redis
Database	PostgreSQL
AI Model	LLM integration — exact provider/model should match project configuration
Practice Platform Focus	HackerRank-style DSA problems

12. Suggested Repository Structure

```

DSA-Coach-HackerRank/
|
+-- app/
|   +-- main.py
|   +-- api/
|   +-- agents/
|   +-- rag/
|   +-- services/
|   +-- database/
|   +-- models/
|
+-- data/
|   +-- problems/
|   +-- knowledge_base/
|
+-- tests/
|
+-- requirements.txt
+-- .env.example
+-- .gitignore
+-- README.md

```

This is a documentation template. Replace names with the repository's actual structure if they differ.

13. Data and Problem Organization

Problems can be categorized using metadata such as difficulty, topic, pattern, expected complexity, and source. A simple logical representation is:

```

{
  "title": "Example Problem",
  "difficulty": "Medium",
  "topics": ["Array", "Hashing"],
  "source": "HackerRank-style",

```

```

    "expected_time": "O(n)",
    "expected_space": "O(n)"
}

```

14. Configuration and Security

Example only – use variables required by the actual project

```

LLM_API_KEY=your_api_key_here
DATABASE_URL=your_postgresql_connection_string
REDIS_URL=your_redis_connection_string
CHROMA_DB_PATH=./chroma_db

```

Security: Never upload real API keys, passwords, tokens, or database credentials to GitHub. Use `.env` locally and provide `.env.example` with placeholder values.

15. Installation and Execution

```

# Clone the repository
git clone <repository-url>
cd DSA-Coach-HackerRank

# Create virtual environment
python -m venv venv

# Windows
venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Configure environment variables
# Create .env from .env.example

# Run FastAPI if this is the project's entry point
uvicorn app.main:app --reload

```

Use the actual entry-point/module name from the repository when running the application.

16. Benefits of the Project

- Structured progression from Simple to Medium to Hard problems.
- Personalized and interactive learning experience.
- Hints encourage independent problem solving.
- AI can explain concepts in learner-friendly language.
- RAG can provide context from the project's curated DSA knowledge.
- Agentic workflow can separate problem analysis, coaching, and feedback.
- Useful for coding-test and technical-interview preparation.

17. Future Enhancements

- Adaptive difficulty based on learner performance.
- Automatic recommendation of the next problem.
- Personalized DSA roadmap.
- Progress tracking and analytics dashboard.

- Code execution in a secure sandbox.
- Automatic test-case generation and explanation.
- Interview mode with timed questions.
- Leaderboard, streaks, badges, and gamification.
- Additional coding-platform integrations.
- Voice-based DSA coaching.

18. Project Outcome

DSA-Coach-HackerRank is intended to provide an AI-assisted environment for structured DSA preparation. By organizing problems into Simple, Medium, and Hard levels and combining guided problem solving with Agentic AI, RAG, vector retrieval, and backend services, the project can help learners move from basic concepts to advanced algorithmic reasoning in a more systematic way.

19. Documentation Accuracy Note

This document uses the project name and scope provided by the team and the architecture components visible in the earlier project documentation screenshots. The exact LLM provider/model, agent names, API routes, database tables, problem dataset, and repository file structure should be synchronized with the actual source code before final submission.